

Федеральное государственное автономное образовательное
учреждение высшего образования «**Национальный
исследовательский университет ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №5
по дисциплине «Вычислительная математика»

Вариант: 9

Преподаватель: Наумова
Надежда Александровна

Выполнил: Ровкова Анастасия Сергеевна
Группа: Р3216

Санкт-Петербург, 2026 г

Цель работы

решить задачу интерполяции, найти значения функции при заданных значениях аргумента, отличных от узловых точек. на языке программирования SCALA.

Рабочие формулы используемых методов.

Многочлен Лагранжа

$$L_n(x) = \sum_{i=0}^n y_i l_i(x)$$

$$L_n(x) = \sum_{i=0}^n y_i \frac{(x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)}$$

$$L_n(x) = \sum_{i=0}^n y_i \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \quad (8)$$

Интерполяционный многочлен Ньютона с разделенными разностями первого вида (вперед):

$$N_n(x) = f(x_0) + f(x_0, x_1) \cdot (x - x_0) + f(x_0, x_1, x_2) \cdot (x - x_0) \cdot (x - x_1) + \dots + f(x_0, x_1, \dots, x_n) \cdot (x - x_0) \cdot (x - x_1) \dots (x - x_{n-1})$$

Интерполяционный многочлен Ньютона с разделенными разностями второго вида (назад):

$$N_n(x) = f(x_n) + f(x_{n-1}, x_n) \cdot (x - x_n) + f(x_{n-2}, x_{n-1}, x_n) \cdot (x - x_n) \cdot (x - x_{n-1}) + \dots + f(x_0, x_1, \dots, x_n) \cdot (x - x_n) \cdot (x - x_{n-1}) \dots (x - x_1)$$

$$N_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!} \Delta^2 y_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!} \Delta^n y_0$$

Первая интерполяционная формула Гаусса ($x > a$)

$$\begin{aligned} P_n(x) &= y_0 + t\Delta y_0 + \frac{t(t-1)}{2!} \Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!} \Delta^3 y_{-1} \\ &+ \frac{(t+1)t(t-1)(t-2)}{4!} \Delta^4 y_{-2} + \frac{(t+2)(t+1)t(t-1)(t-2)}{5!} \Delta^5 y_{-2} \dots \\ &+ \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!} \Delta^{2n-1} y_{-(n-1)} + \frac{(t+n-1)\dots(t-n)}{(2n)!} \Delta^{2n} y_{-n} \end{aligned}$$

$$\begin{aligned} P_n(x) &= y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!} \Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!} \Delta^3 y_{-2} \\ &+ \frac{(t+2)(t+1)t(t-1)}{4!} \Delta^4 y_{-2} + \dots + \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!} \Delta^{2n-1} y_{-n} \\ &+ \frac{(t+n)(t+n-1)\dots(t-n+1)}{(2n)!} \Delta^{2n} y_{-n} \end{aligned}$$

Многочлен Стирлинга

$$\begin{aligned} P_n(x) &= y_0 + t \frac{\Delta y_{-1} + \Delta y_0}{2!} + \frac{t^2}{2} \Delta^2 y_{-1} + \frac{t(t^2 - 1^2)}{3!} \cdot \frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} + \frac{t^2(t^2 - 1^2)}{4!} \Delta^4 y_{-2} \\ &+ \frac{t(t^2 - 1^2)(t^2 - 2^2)}{5!} \cdot \frac{\Delta^5 y_{-3} + \Delta^5 y_{-2}}{2} + \frac{t(t^2 - 1^2)(t^2 - 2^2)}{6!} \cdot \Delta^6 y_{-3} + \dots \\ &+ \frac{t(t^2 - 1^2)(t^2 - 2^2)\dots(t^2 - (n-1)^2)}{(2n-1)!} \cdot \frac{\Delta^{2n-1} y_{-n} + \Delta^{2n-1} y_{-(n-1)}}{2} + \\ &+ \frac{t^2(t^2 - 1^2)(t^2 - 2^2)\dots(t^2 - (n-1)^2)}{(2n)!} \cdot \Delta^{2n} y_{-n} \end{aligned}$$

Многочлен Бесселя

$$\begin{aligned}
P_n(x) &= \frac{y_0 + y_1}{2} + \left(t - \frac{1}{2}\right) \Delta y_0 + \frac{t(t-1)}{2!} \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \frac{\left(t - \frac{1}{2}\right) t(t-1)}{3!} \Delta^3 y_{-1} \\
&+ \frac{t(t-1)(t+1)(t-2)}{4!} \frac{\Delta^4 y_{-2} + \Delta^4 y_{-1}}{2} + \frac{\left(t - \frac{1}{2}\right) t(t-1)(t+1)(t-2)}{5!} \Delta^5 y_{-2} \\
&+ \frac{t(t-1)(t+1)(t-2)(t+2)(t-3)}{6!} \frac{\Delta^6 y_{-3} + \Delta^6 y_{-2}}{2} + \dots \\
&+ \frac{t(t-1)(t+1)(t-2)(t+2) \dots (t-n)(t+n-1)}{(2n)!} \cdot \frac{\Delta^{2n} y_{-n} + \Delta^{2n} y_{-(n-1)}}{2} \\
&+ \frac{\left(t - \frac{1}{2}\right) t(t-1)(t+1)(t-2)(t+2) \dots (t-n)(t+n-1)}{(2n+1)!} \Delta^{2n+1} y_{-n}
\end{aligned}$$

Вычислительная часть.

По варианту 9.

х	у
1,05	0,1213
1,15	1,1316
1,25	2,1459
1,35	3,1565
1,45	4,1571
1,55	5,1819
1,65	6,1969

Заданные точки интерполирования:

$$X_1 = 1,562$$

$$X_2 = 1,362$$

Шаг таблицы:

$$h = 0,1$$

Так как узлы таблицы расположены равномерно, можно использовать конечные разности, формулы Ньютона и формулы Гаусса.

Таблица конечных разностей

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0	1,05	0,1213	1,0103	0,0040	-0,0077	0,0014	0,0391	-0,1478
1	1,15	1,1316	1,0143	-0,0037	-0,0063	0,0405	-0,1087	—
2	1,25	2,1459	1,0106	-0,0100	0,0342	-0,0682	—	—
3	1,35	3,1565	1,0006	0,0242	-0,0340	—	—	—
4	1,45	4,1571	1,0248	-0,0098	—	—	—	—
5	1,55	5,1819	1,0150	—	—	—	—	—
6	1,65	6,1969	—	—	—	—	—	—

Вычисление значения в точке $X_1 = 1,562$ по формуле

Ньютона Так как $X_1 = 1,562$ находится ближе к концу таблицы, используем вторую интерполяционную формулу Ньютона, то есть формулу назад.

Берём последний узел:

$$x_n = 1,65$$

$$y_n = 6,1969$$

$$t = (X_1 - x_n) / h = (1,562 - 1,65) / 0,1 = -0,88$$

Формула Ньютона назад:

$$P(x) = y_n + t \nabla y_n + t(t+1)/2! \nabla^2 y_n + t(t+1)(t+2)/3! \nabla^3 y_n + \dots$$

Используем последние конечные разности:

$$\nabla y_n = 1,0150$$

$$\nabla^2 y_n = -0,0098$$

$$\nabla^3 y_n = -0,0340$$

$$\nabla^4 y_n = -0,0682$$

$$\nabla^5 y_n = -0,1087$$

$$\nabla^6 y_n = -0,1478$$

После подстановки:

$$P(1,562) \approx 5,306970$$

Ответ:

$$f(1,562) \approx 5,306970$$

Вычисление значения в точке $X_2 = 1,362$ по Гаусса

Точка $X_2 = 1,362$ находится около середины таблицы, поэтому используем первую формулу Гаусса.

В качестве центрального узла выбираем:

$$x_0 = 1,35$$

$$y_0 = 3,1565$$

$$t = (X_2 - x_0) / h = (1,362 - 1,35) / 0,1 = 0,12$$

Формула Гаусса вперед:

$$P(x) = y_0 + t\Delta y_0 + t(t-1)/2! \Delta^2 y_{-1} + (t+1)t(t-1)/3! \Delta^3 y_{-1} + \dots$$

Подставляем значения из таблицы конечных

разностей: $y_0 = 3,1565$

$$\Delta y_0 = 1,0006$$

$$\Delta^2 y_{-1} = -0,0100$$

$$\Delta^3 y_{-1} = 0,0342$$

$$\Delta^4 y_{-2} = 0,0405$$

$$\Delta^5 y_{-2} = -0,1087$$

$$\Delta^6 y_{-3} = -0,1478$$

После

подстановки:

$$P(1,362) \approx 3,276653$$

Ответ:

$$f(1,362) \approx 3,276653$$

Лангранж:

Многочлен Лагранжа имеет вид:

$$L(x) = \sum y_i \cdot l_i(x)$$

где

$$l_i(x) = \prod (x - x_j) / (x_i - x_j), \text{ при } j \neq i.$$

Для заданной таблицы были вычислены значения в точках X_1 и X_2 . Полученные результаты:

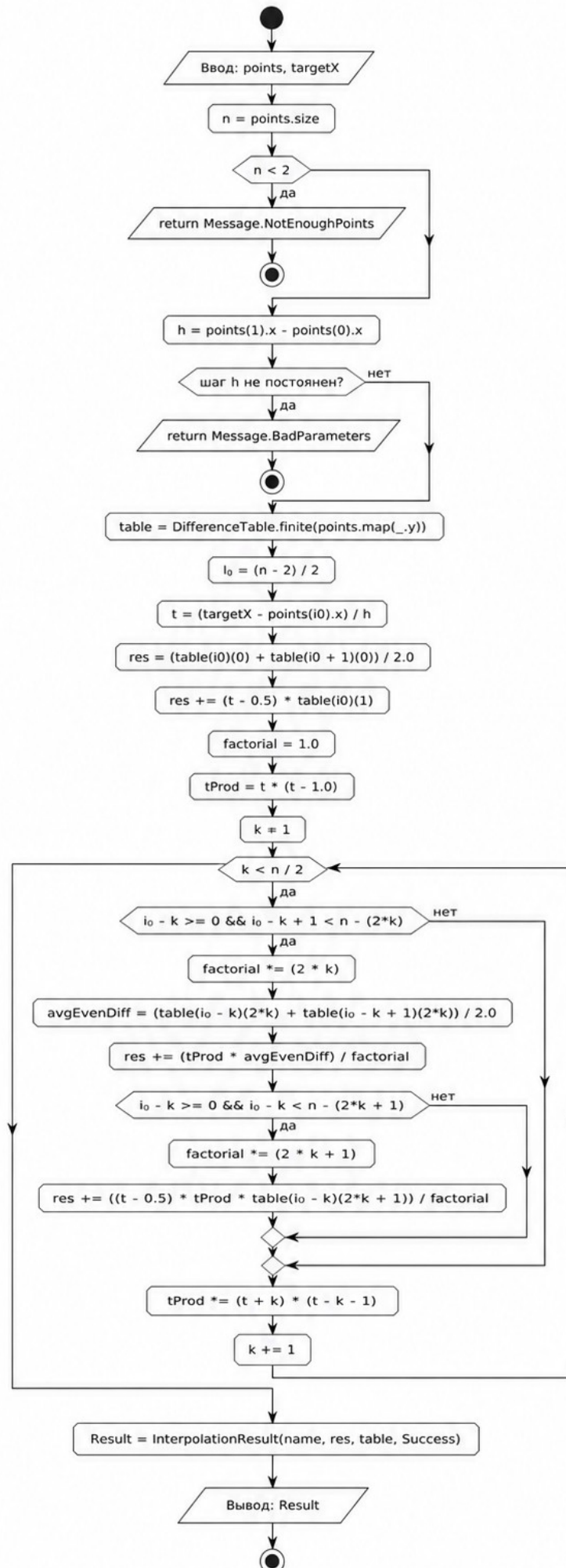
$$L(1,562) \approx 5,306970$$

$$L(1,362) \approx 3,276653$$

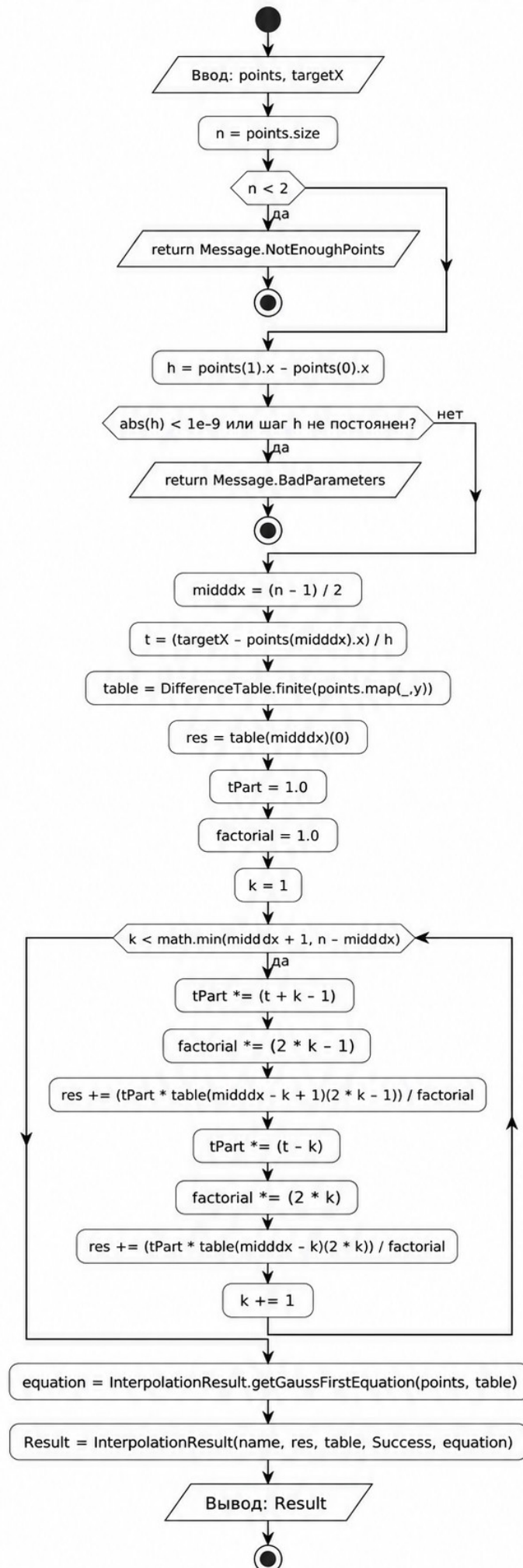
Сравнение результатов

Блок схемы

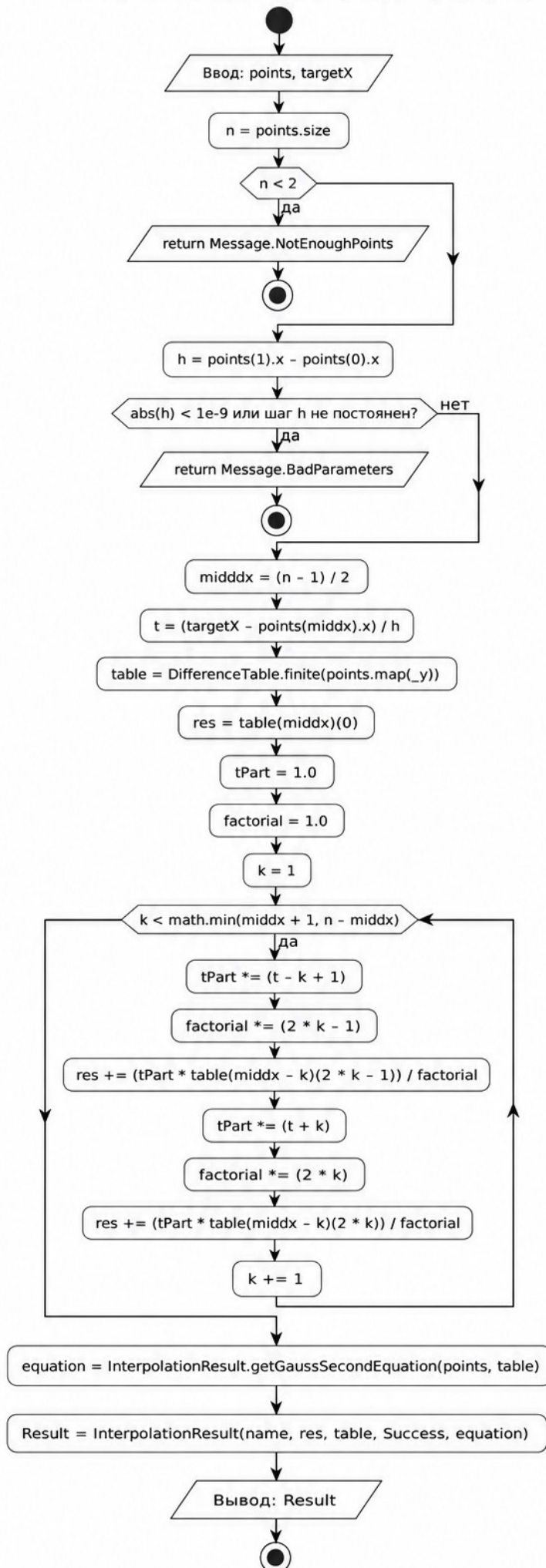
Схема Бесселя



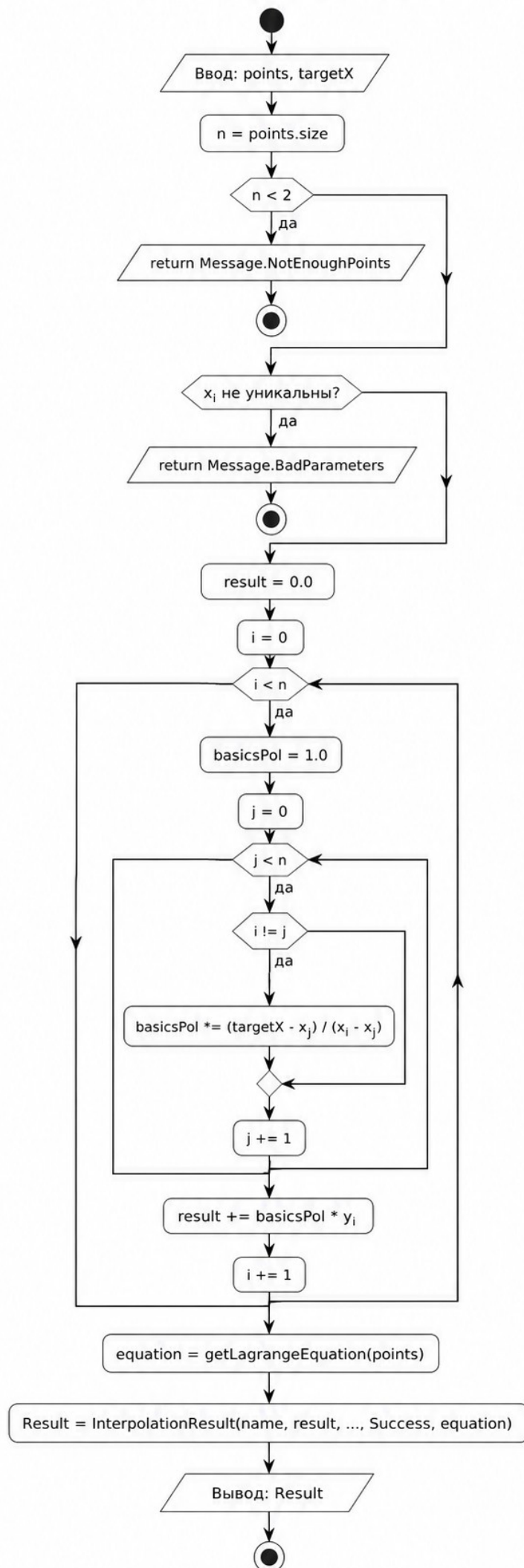
Алгоритм метода Гаусса (первая формула)



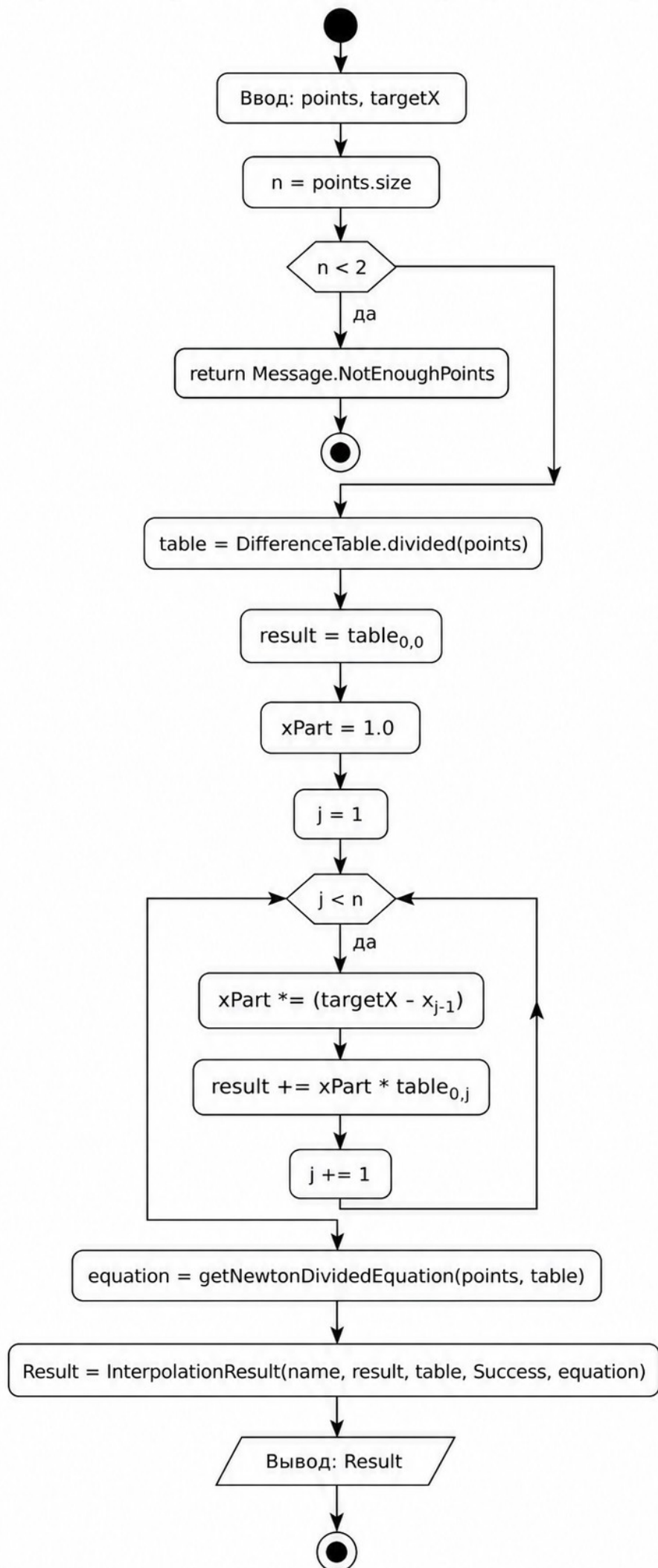
Алгоритм метода Гаусса (вторая формула)



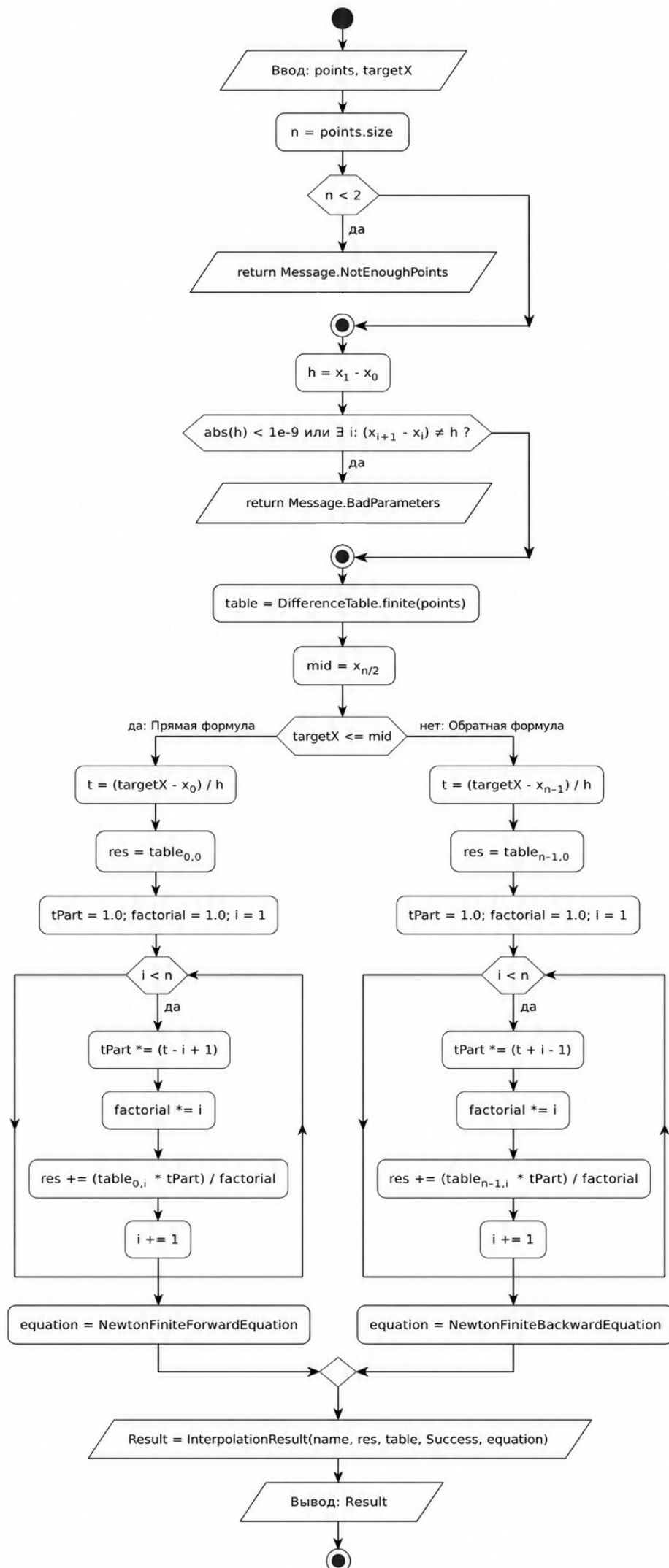
Алгоритм интерполяции Лагранжа



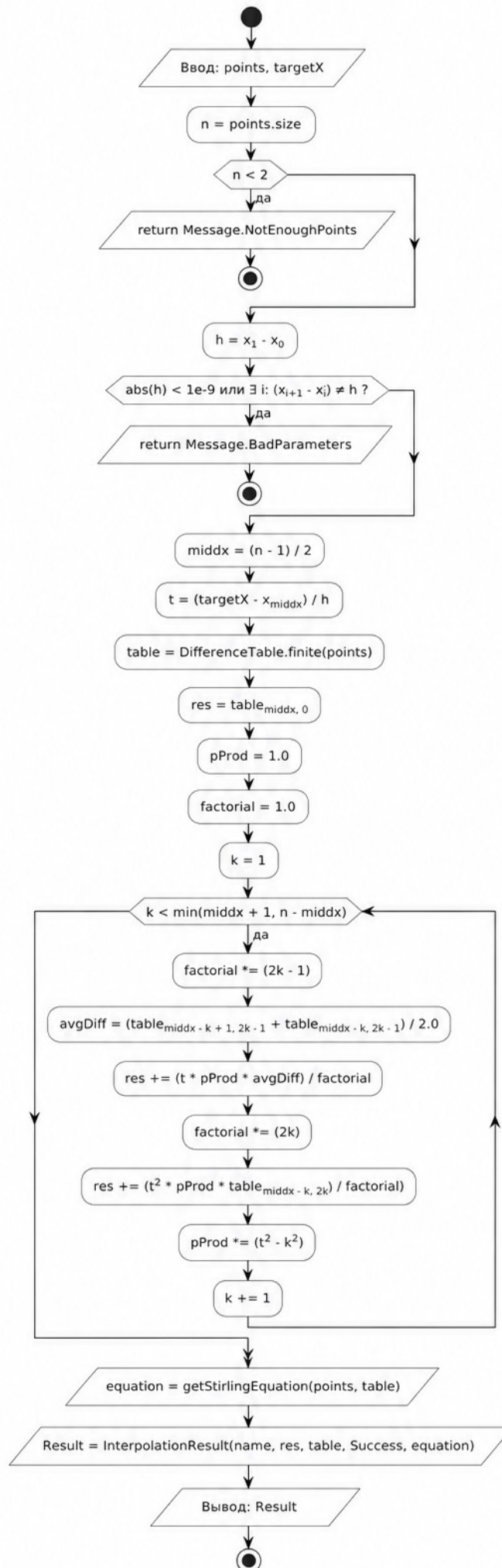
Алгоритм интерполяции Ньютона (разделенные разности)



Алгоритм интерполяции Ньютона (для равноотстоящих узлов)



Алгоритм интерполяции Стирлинга



Классы, реализующие методы. package

solver.interpolation.methods

```
import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point,
InterpolationResult} import
solver.interpolation.core.DifferenceTable
import solver.core.Message

class Bessel extends Interpolator {
  override def name: String = "Схема Бесселя"

  override def solve(points: Seq[Point], targetX: Double):
    InterpolationResult = { val n = points.size
    if (n < 2) return InterpolationResult(name, 0, Seq.empty,
Message.NotEnoughPoints)

    val h = points(1).x - points(0).x
    if (!points.sliding(2).forall(p => math.abs((p(1).x - p(0).x) - h) <
    1e-9)) return InterpolationResult(name, 0, Seq.empty,
    Message.BadParameters)

    val table =

    DifferenceTable.finite(points.map(_.y)) var i0

    = points.indexWhere(_.x > targetX) - 1

    if (i0 < 0) i0 = 0
    if (i0 > n - 2) i0 = n - 2

    val t = (targetX - points(i0).x) / h

    var res = (table(i0)(0) + table(i0 + 1)(0)) /
    2.0 res += (t - 0.5) * table(i0)(1)

    var factorial = 1.0
    var tProd = t * (t -
    1.0)
```

```

for (k <- 1 until n / 2) {
  val evenOrder = 2 *
  k

  if (i0 - k >= 0 && i0 - k + 1 < n - evenOrder)
    { factorial *= evenOrder
      val avgEvenDiff = (table(i0 - k)(evenOrder) + table(i0 - k + 1)
        (evenOrder)) / 2.0 res += (tProd * avgEvenDiff) / factorial
      val oddOrder = 2 * k + 1
      if (i0 - k >= 0 && i0 - k < n -
        oddOrder) { factorial *= oddOrder
        res += ((t - 0.5) * tProd * table(i0 - k)(oddOrder)) / factorial
        }
      }

  tProd *= (t + k) * (t - k - 1)
}

InterpolationResult(name, res, table, Message.Success)
}
}

```

package solver.interpolation.methods

```

import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point, InterpolationResult}
import solver.interpolation.core.DifferenceTable
import solver.core.Message

class GaussSecond extends Interpolator {
  override def name: String = "Многочлен Гаусса (вторая формула)"

  override def solve(points: Seq[Point], targetX: Double): InterpolationResult = {
    val n = points.size
    if (n < 2) {
      return InterpolationResult(name, 0.0, Seq.empty, Message.NotEnoughPoints)
    }
  }
}

```

```

val h = points(1).x - points(0).x
if (math.abs(h) < 1e-9) {
  return InterpolationResult(name, 0.0, Seq.empty, Message.BadParameters)
}

if (!points.sliding(2).forall(p => math.abs((p(1).x - p(0).x) - h) < 1e-9))
  return InterpolationResult(name, 0.0, Seq.empty, Message.BadParameters)

val midIdx = (n - 1) / 2
val t = (targetX - points(midIdx).x) / h
val table = DifferenceTable.finite(points.map(_.y))

var res = table(midIdx)(0)
var tPart = 1.0
var factorial = 1.0

for (k <- 1 until math.min(midIdx + 1, n - midIdx)) {
  tPart *= (t - k + 1)
  factorial *= (2 * k - 1)
  res += (tPart * table(midIdx - k)(2 * k - 1)) / factorial

  tPart *= (t + k)
  factorial *= (2 * k)
  res += (tPart * table(midIdx - k)(2 * k)) / factorial
}
InterpolationResult(name, res, table, Message.Success, equation =
InterpolationResult.getGaussSecondEquation(points, table))
}
}

```

package solver.interpolation.methods

```

import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point, InterpolationResult}
import solver.interpolation.core.DifferenceTable
import solver.core.Message

class GaussFirst extends Interpolator {
  override def name: String = "Многочлен Гаусса (первая формула)"

  override def solve(points: Seq[Point], targetX: Double): InterpolationResult = {
    val n = points.size
    println("GaussFirst")
    if (n < 2) {
      return InterpolationResult(name, 0.0, Seq.empty, Message.NotEnoughPoints)
    }
  }
}

```

```

}

val h = points(1).x - points(0).x
if (math.abs(h) < 1e-9) {
  return InterpolationResult(name, 0.0, Seq.empty, Message.BadParameters)
}

if (!points.sliding(2).forall(p => math.abs((p(1).x - p(0).x) - h) < 1e-9))
  return InterpolationResult(name, 0.0, Seq.empty, Message.BadParameters)

val midIdx = (n - 1) / 2
val t = (targetX - points(midIdx).x) / h
val table = DifferenceTable.finite(points.map(_.y))

var res = table(midIdx)(0)
var tPart = 1.0
var factorial = 1.0

for (k <- 1 until math.min(midIdx + 1, n - midIdx)) {
  tPart *= (t + k - 1)
  factorial *= (2 * k - 1)
  res += (tPart * table(midIdx - k + 1)(2 * k - 1)) / factorial

  tPart *= (t - k)
  factorial *= (2 * k)
  res += (tPart * table(midIdx - k)(2 * k)) / factorial
}
InterpolationResult(name, res, table, Message.Success, equation =
InterpolationResult.getGaussFirstEquation(points, table))
}
}

```

package solver.interpolation.methods

```

import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point,
InterpolationResult} import solver.core.Message

class Lagrange extends Interpolator {
  override def name: String = "Многочлен Лагранжа"

  override def solve(points: Seq[Point], targetX: Double):

```

```
InterpolationResult = { val n = points.size
```

```
if (n < 2) {  
  return InterpolationResult(name, 0, Seq.empty,  
    Message.NotEnoughPoints)  
}
```

```
if (points.map(_.x).distinct.size != n) {  
  return InterpolationResult(name, 0, Seq.empty,  
    Message.BadParameters)  
}
```

```
var result = 0.0
```

```
for (i <- 0 until n) {  
  var basicsPol =  
    1.0 for (j <- 0  
      until n) { if (i != j)  
        {  
          basicsPol *= (targetX - points(j).x) / (points(i).x - points(j).x)  
        }  
      }  
  result += basicsPol * points(i).y  
}
```

```
InterpolationResult(  
  methodName =  
  name, value =  
  result,  
  differenceTable =  
  Seq(points.map(_.y)), message =  
  Message.Success  
)  
}
```

```
}package solver.interpolation.methods
```

```
import solver.interpolation.functions.Interpolator  
import solver.interpolation.models.{Point,  
InterpolationResult} import
```

```

solver.interpolation.core.DifferenceTable
import solver.core.Message

class NewtonDivided extends Interpolator {
  override def name: String = "Ньютон (раздел. разности)"

  override def solve(points: Seq[Point], targetX: Double):
    InterpolationResult = { val n = points.size
    if (n < 2) return InterpolationResult(name, 0, Seq.empty,
    Message.NotEnoughPoints)

    val table =

    DifferenceTable.divided(points) var

    result = table(0)(0)
    var xPart = 1.0

    for (j <- 1 until n) {
      xPart *= (targetX - points(j -
      1).x) result += xPart * table(0)
      (j)
    }

    InterpolationResult( met
      hodName = name,
      value = result,
      differenceTable = table,
      message =
      Message.Success
    )
  }
}

```

package solver.interpolation.methods

```
import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point,
InterpolationResult} import
solver.interpolation.core.DifferenceTable
import solver.core.Message
```

```
class NewtonFinite extends Interpolator {
  override def name: String = "Ньютон (конечн. разности)"
```

```
  override def solve(points: Seq[Point], targetX: Double):
```

```
    InterpolationResult = { val n = points.size
```

```
    if (n < 2) {
```

```
      return InterpolationResult(name, 0.0, Seq.empty,
        Message.NotEnoughPoints)
```

```
    }
```

```
    val h = points(1).x -
```

```
    points(0).x if (math.abs(h)
```

```
    < 1e-9) {
```

```
      return InterpolationResult(name, 0.0, Seq.empty,
        Message.BadParameters)
```

```
    }
```

```
    val isEquidistant = points.sliding(2).forall(p => math.abs((p(1).x - p(0).x) -
```

```
    h) < 1e-9) if (!isEquidistant) {
```

```
      return InterpolationResult(name, 0.0, Seq.empty,
        Message.BadParameters)
```

```
    }
```

```
    val table =
```

```
    DifferenceTable.finite(points.map(_._y)) val
```

```
    mid = points(n/2).x
```

```
    if (targetX <= mid) {
```

```
      val t = (targetX - points(0).x) /
```

```
      h var res = table(0)(0)
```

```
      var tPart = 1.0
```

```
      var factorial =
```

1.0

```
for (i <- 1 until n) {
  tPart *= (t - i + 1)
  factorial *= i
  res += (table(0)(i) * tPart) / factorial
}
InterpolationResult(name, res, table, Message.Success)
} else {
  val t = (targetX - points.last.x) / h
  var res = table.last(0) var tPart = 1.0
  var factorial = 1.0

  for (i <- 1 until n) {
    tPart *= (t + i - 1)
    factorial *= i
    res += (table(n - 1 - i)(i) * tPart) / factorial
  }
  InterpolationResult(name, res, table, Message.Success)
}
}
```

```
package solver.interpolation.methods
```

```
import solver.interpolation.functions.Interpolator
import solver.interpolation.models.{Point,
InterpolationResult} import
solver.interpolation.core.DifferenceTable
import solver.core.Message
```

```
class Stirling extends Interpolator {
  override def name: String = "Схема Стирлинга"

  override def solve(points: Seq[Point], targetX: Double):
```

```

InterpolationResult = { val n = points.size
if (n < 2) {
    return InterpolationResult(name, 0.0, Seq.empty,
    Message.NotEnoughPoints)
}

val h = points(1).x -
points(0).x if (math.abs(h)
< 1e-9) {
    return InterpolationResult(name, 0.0, Seq.empty,
    Message.BadParameters)
}

val isEquidistant = points.sliding(2).forall(p => math.abs((p(1).x - p(0).x) -
h) < 1e-9) if (!isEquidistant) {
    return InterpolationResult(name, 0.0, Seq.empty,
    Message.BadParameters)
}

val midIdx = points.zipWithIndex.minBy(p => math.abs(p._1.x - targetX))._2 val t =
(targetX - points(midIdx).x) / h
val table = DifferenceTable.finite(points.map(_._2))

var res = table(midIdx)
(0) var pProd = 1.0
var factorial = 1.0

for (k <- 1 until math.min(midIdx + 1, n -
midIdx)) { factorial *= (2 * k - 1)
    val avgDiff = (table(midIdx - k + 1)(2 * k - 1) + table(midIdx - k)(2 * k -
1)) / 2.0 res += (t * pProd * avgDiff) / factorial

    factorial *= (2 * k)
    res += (t * t * pProd * table(midIdx - k)(2 * k)) / factorial

    pProd *= (t * t - k * k)
}

InterpolationResult(name, res, table, Message.Success)
}
}

```

Листинг программы

https://github.com/Naswek/ITMO_labs/tree/main/
%D0%92%D1%8B%D1%87%D0%BC%D0%B0%D1%82/%D0%BB%D0%B0
%D0%B1%202/backend/src/main/scala/solver/interpolation

Результаты выполнения программы при различных исходных данных.

Лаб 2 (Уравнения) Лаб 3 (Интегралы) Лаб 4 (Аппроксимации) Лаб 5 (Интерполяции)

Интерполяция функций (Лагранж, Ньютон, Гаусс, Стирлинг, Бессель)

Настройки интерполяции

Из функции

Ввод таблицы

Выберите функцию

cos(2*x)

Начало (a) 0

Вариант (b) 2

Шаг (h) 0.2

Загрузить параметры (id a b h targetX)

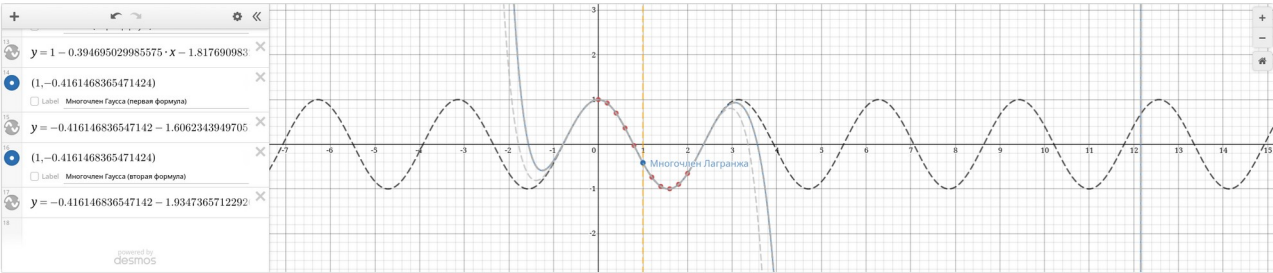
Точка интерполирования (X) 1

Итого точек: 11

Вычислить значения

Результаты интерполирования

Метод	Уравнение	Вычисленное значение f(X)	Статус	Разности
Многочлен Лагранжа	-0.4161468365471424	Успешно	Посчитать	
Схема Стирлинга	-0.4161468365471424	Успешно	Посчитать	
Ньютон (первая формула)	-0.4161468365471425	Успешно	Посчитать	
Схема Бесселя	-0.41614683654714235	Успешно	Посчитать	
Ньютон (вторая формула)	-0.4161468365471424	Успешно	Посчитать	
Многочлен Гаусса (первая формула)	-0.4161468365471424	Успешно	Посчитать	
Многочлен Гаусса (вторая формула)	-0.4161468365471424	Успешно	Посчитать	



Чёрный пунктир — идеальная функция (если сгенерирована). Красные точки — узлы интерполяции. Оранжевый пунктир — целевой X. Синие точки — интерполированные значения.

Лаб 2 (Уравнения) Лаб 3 (Интегралы) Лаб 4 (Аппроксимации) Лаб 5 (Интерполяции)

Интерполяция функций (Лагранж, Ньютон, Гаусс, Стирлинг, Бессель)

Настройки интерполяции

Из функции

Ввод таблицы

Выберите функцию

2*exp(0.5*x)+1

Начало (a) 0

Вариант (b) 2

Шаг (h) 0.2

Загрузить параметры (id a b h targetX)

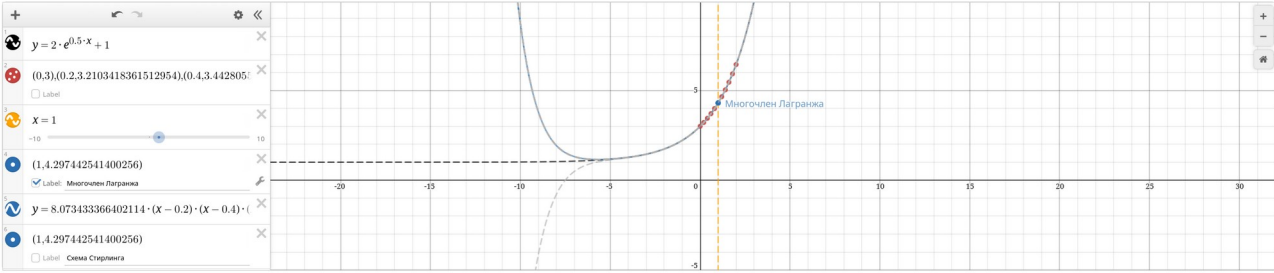
Точка интерполирования (X) 1

Итого точек: 11

Вычислить значения

Результаты интерполирования

Метод	Уравнение	Вычисленное значение f(X)	Статус	Разности
Многочлен Лагранжа	4.297442541400256	Успешно	Посчитать	
Схема Стирлинга	4.297442541400256	Успешно	Посчитать	
Ньютон (первая формула)	4.297442541400256	Успешно	Посчитать	
Схема Бесселя	4.297442541400256	Успешно	Посчитать	
Ньютон (вторая формула)	4.297442541400256	Успешно	Посчитать	
Многочлен Гаусса (первая формула)	4.297442541400256	Успешно	Посчитать	
Многочлен Гаусса (вторая формула)	4.297442541400256	Успешно	Посчитать	



Чёрный пунктир — идеальная функция (если сгенерирована). Красные точки — узлы интерполяции. Оранжевый пунктир — целевой X. Синие точки — интерполированные значения.

Интерполяция функций (Лагранж, Ньютон, Гаусс, Стирлинг, Бессель)

Настройки интерполяции

Из функции

Ввод таблицы

Выберите функцию

3*ln(x)+2

Начало (a)

Конец (b)

Шаг (h)

0.1

2

0.2

Загрузить параметры (id a b h targetX)

Точка интерполирования (X)

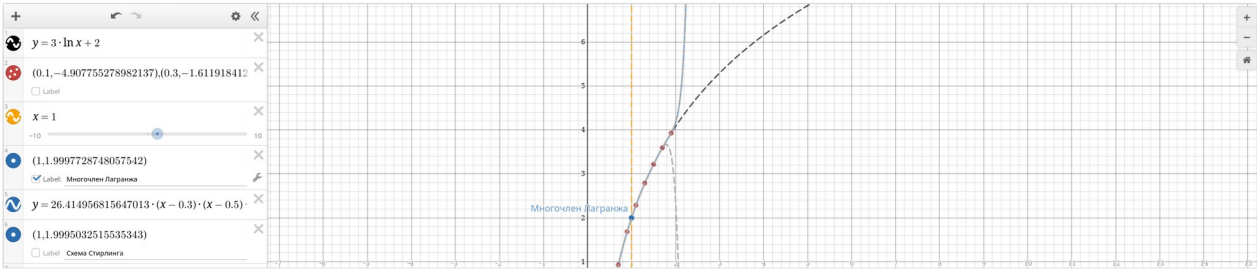
1

Итого точек: 10

Вычислить значения

Результаты интерполирования

Метод	Уравнение	Вычисленное значение f(X)	Статус	Разности
Многочлен Лагранжа	1.9997728748057542	Успешно	Показать	
Схема Стирлинга	1.9995032515535343	Успешно	Показать	
Ньютон (первая формула)	1.999772874805752	Успешно	Показать	
Схема Бесселя	1.9997728748057542	Успешно	Показать	
Ньютон (вторая формула)	1.9997728748057548	Успешно	Показать	
Многочлен Гаусса (первая формула)	1.999503251553534	Успешно	Показать	
Многочлен Гаусса (вторая формула)	1.9995032515535334	Успешно	Показать	



Вывод

В результате выполнения лабораторной работы были изучены и реализованы основные методы интерполяции функций. Проведённые вычисления и тестирование программы показали корректность работы реализованных алгоритмов на различных наборах данных на языке программирования SCALA.